

JavaScript Object Definition

[Back to JS page](#)

Table of Contents

- [JavaScript Object Definition](#)
 - [Using an Object Literal](#)
 - [Example 1](#)
 - [Using the new Keyword](#)
 - [Example 2](#)
 - [JavaScript Object.create\(\).](#)
 - [Example 3](#)
 - [JavaScript Object.assign\(\).](#)
 - [Example 4](#)
 - [JavaScript Objects are Mutable](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

Using an Object Literal

The easiest way to create a JavaScript object is using an **object literal**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using an Object Literal</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  eyeColor: "blue"
};

document.getElementById("demo").innerHTML =
  person.firstName + " " + person.lastName + ", age " + person.age;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Using the new Keyword

You can create an object using the `new Object()` constructor:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using the new Keyword</h4>
<p id="demo"></p>

<script>
const person = new Object();
person.firstName = "John";
person.lastName = "Doe";
person.age = 30;

document.getElementById("demo").innerHTML =
    person.firstName + " " + person.lastName + ", age " + person.age;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Object.create()

The `Object.create()` method creates a new object using an existing object as a prototype:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Object.create()</h4>
<p id="demo"></p>

<script>
// Define prototype object
const personProto = {
  greet: function() {
    return "Hello, I'm " + this.firstName;
  }
};

// Create new object based on prototype
const person = Object.create(personProto);
person.firstName = "John";
person.lastName = "Doe";

document.getElementById("demo").innerHTML = person.greet();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

JavaScript Object.assign()

The `Object.assign()` method copies properties from one or more source objects to a target object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Object.assign()</h4>
<p id="demo"></p>

<script>
const person = { firstName: "John", lastName: "Doe" };
const address = { city: "New York", country: "USA" };
const contact = { email: "john@example.com" };

// Merge objects into a new object
const merged = Object.assign({}, person, address, contact);

document.getElementById("demo").innerHTML =
  merged.firstName + " " + merged.lastName +
  ", " + merged.city + ", " + merged.country +
  "<br>Email: " + merged.email;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

JavaScript Objects are Mutable

Objects are mutable: they are accessed by reference, not by value.

If you change a property of a copied object, the original object is also changed:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Definition</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Objects are Mutable</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const person = { firstName: "John", lastName: "Doe" };

// Copy by reference (not by value)
const employee = person;
employee.firstName = "Jane";

document.getElementById("demo1").innerHTML = "person: " + person.firstName;
document.getElementById("demo2").innerHTML = "employee: " + employee.firstName;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Object Definition](#)